SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO5: *Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques.*
(BL : 3)

Assessment Tool Weightage: 10%

QUESTIONS: 20 Total Marks:20 Duration :1 Hour

Mock Interview question

Code of Conduct:

I Yuvann Nithish R(192411267) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Yuvann Nithish R

SIMATS ENGINEERING
ASSESSMENT TOOLS

1. What is a Pushdown Automaton (PDA)? How is it different from a Finite Automaton? 2. Define the components of a PDA formally.
3. What is a stack in PDA? Why is it important?

4. What is an Instantaneous Description (ID) in PDA?

5. What are the different types of moves in a PDA?

6. What is acceptance by final state and empty stack?

7. Define the language accepted by a PDA.

8. How does a PDA process a string using IDs?

9. Differentiate between Deterministic PDA and Non-Deterministic PDA. 10. Why are PDAs equivalent to Context-Free Grammars (CFG)?
11. What types of languages are accepted by PDA? Give examples.

12. What is a Turing Machine? How is it more powerful than PDA?

13. Define the components of a Turing Machine.

14. Explain tape, head, and state transitions in a Turing Machine.

15. Explain programming techniques for Turing Machines.

16. What is storage in finite control in Turing Machines?

17. Compare FA, PDA, and TM based on memory, power, and language acceptance. 18. What is DFA?
19. Which language is accepted by a Turing Machine?

20.How turing machines are applied in real world applications?

PDA & Turing Machine — Important Questions and Answers

1. What is a Pushdown Automaton (PDA)? How is it different from a Finite Automaton?

A **Pushdown Automaton (PDA)** is a computational model used to recognize **Context-Free Languages (CFLs)**. It is essentially a finite automaton equipped with an additional **stack**.

Difference

Feature	Finite Automaton	PDA
Memory	No stack	Has a stack
Storage	Finite control only	Stack + finite control
Language	Regular languages	Context-Free languages
Example	(a^*)	(a^n b^n)
Power	Less powerful	More powerful

A PDA can recognize:

$$
L=\{a^n b^n\mid n\geq0\}
$$

because it can store the number of as in its stack.

---

2. Define the components of a PDA formally.

A PDA is formally represented as:

$$
M=(Q,\Sigma,\Gamma,\delta,q_0,Z_0,F)
$$

where:

-

(Q) = finite set of states

-
-

(\Sigma) = input alphabet

-
-

(\Gamma) = stack alphabet

-
-

(\delta) = transition function

-
-

(q_0) = initial state

-
-

(Z_0) = initial stack symbol

-
-

(F) = set of final/accepting states

-

For a nondeterministic PDA:

$$
\delta:Q\times(\Sigma\cup\{\epsilon\})\times\Gamma
\rightarrow
\mathcal P(Q\times\Gamma^*)
$$

---

3. What is a stack in PDA? Why is it important?

A **stack** is an unbounded memory structure used by a PDA.

It follows the:

$$
\boxed{LIFO\text{ — Last In, First Out}}
$$

principle.

Operations

- **Push** → add a symbol
- **Pop** → remove the top symbol
- **Read** → inspect the top symbol

Importance

The stack allows a PDA to remember an arbitrary amount of information.

For example, for:

$$
a^n b^n
$$

the PDA pushes one symbol for every `a` and pops one symbol for every `b`.

---

4. What is an Instantaneous Description (ID) in PDA?

An **Instantaneous Description (ID)** represents the current configuration of a PDA.

It is generally written as:

[
(q,w,\gamma)
]

where:

- (q) = current state
- (w) = remaining input
- (\gamma) = current stack contents

Example

[
(q_1,abb,AAB)
]

means:

- Current state = (q_1)
- Remaining input = `abb`

Stack = `AAB`

-

The symbol at the top of the stack is usually taken as the leftmost/rightmost symbol depending on the notation convention.

---

5. What are the different types of moves in a PDA?

A PDA can perform several types of moves.

1. Input-consuming move

The PDA reads an input symbol and changes its state/stack.

Example:

$$
\delta(q,a,A)=(p,AA)
$$

It reads `a` and replaces (A) with (AA).

2. Push move

A symbol is added to the stack.

Example:

$$
A\rightarrow BA
$$

3. Pop move

The top stack symbol is removed.

Example:

$$
A\rightarrow\epsilon
$$

4. Epsilon move

The PDA changes state or stack **without consuming an input symbol**.

Example:

$$
\delta(q,\epsilon,A)=(p,B)
$$

---

6. What is acceptance by final state and empty stack?

A PDA can accept a string in two common ways.

Acceptance by final state

A string is accepted if, after consuming the input, the PDA reaches a final state.

$$
(q_0,w,Z_0)\vdash^*(q_f,\epsilon,\gamma)
$$

where:

$$
q_f\in F
$$

The stack does not necessarily have to be empty.

Acceptance by empty stack

A string is accepted if the entire input is consumed and the stack becomes empty.

$$
(q_0,w,Z_0)\vdash^*(q,\epsilon,\epsilon)
$$

Difference

Final State	Empty Stack
Final state must be reached	Stack must become empty

Final State	Empty Stack
Stack may contain symbols	No stack symbols remain
Both have equivalent power for NPDAs	Both have equivalent power for NPDAs

---

7. Define the language accepted by a PDA.

If (M) is a PDA, the language accepted by final state is:

[
L(M)={w\in\Sigma^*mid(q_0,w,Z_0)\vdash^(q_f,\epsilon,\gamma),q_f\in F}
]

In simple words:

> The language of a PDA is the set of all strings that the PDA accepts.

For example:

[
L={a^n b^n\mid n\geq0}
]

can be accepted by a PDA.

---

8. How does a PDA process a string using IDs?

The PDA processes the input by repeatedly changing its **Instantaneous Description**.

Consider:

[
L={a^n b^n}
]

and input:

[
aabb
]

A simplified sequence can be represented as:

[
(q_0,aabb,Z_0)
]

Read `a` and push (A):

[
\vdash(q_0,abb,AZ_0)
]

Read another `a`:

[
\vdash(q_0,bb,AAZ_0)
]

Read `b` and pop:

[
\vdash(q_1,b,AZ_0)
]

Read another `b`:

[
\vdash(q_1,\epsilon,Z_0)
]

Then move to the final state:

[
\vdash(q_f,\epsilon,Z_0)
]

Therefore:

[
\boxed{aabb\text{ is accepted}}
]

---

9. Differentiate between DPDA and NPDA.

DPDA	NPDA
Deterministic PDA	Nondeterministic PDA
At most one possible transition	Multiple transitions may be possible
No guessing	Can make choices
Less powerful	More powerful
Accepts deterministic CFLs	Accepts all CFLs
Used in deterministic parsing	Useful for general CFG recognition

Important relationship:

$$
\boxed{DPDA\subsetneq NPDA}
$$

Every language accepted by a DPDA is context-free, but not every context-free language is accepted by a DPDA.

---

10. Why are PDAs equivalent to CFGs?

PDAs and CFGs are equivalent in **language-generating/recognizing power**.

The fundamental theorem is:

$$
\boxed{L\text{ is context-free}\iff L\text{ is accepted by an NPDA}}
$$

There are two directions:

CFG → PDA

The PDA simulates grammar derivations using its stack.

For example:

$$
S\rightarrow aSb\mid\epsilon
$$

The PDA can push grammar variables and replace them according to productions.

PDA → CFG

Variables can be constructed to represent relationships between PDA states and stack symbols.

Therefore:

$$
\boxed{CFG\equiv NPDA}
$$

in terms of the class of languages they describe.

---

11. What types of languages are accepted by PDA? Give examples.

A PDA accepts **Context-Free Languages (CFLs)**.

Examples

1. Equal numbers in two blocks

$$
L=\{a^n b^n\mid n\geq0\}
$$

2. Balanced parentheses

$$
L=\{w\mid w\text{ contains balanced parentheses}\}
$$

3. Palindromes

A suitable NPDA can recognize:

$$
L=\{ww^R\mid w\in\{0,1\}^*\}
$$

PDA cannot recognize

$$
\{a^n b^n c^n \mid n \geq 1\}
$$

because this language is not context-free.

---

12. What is a Turing Machine? How is it more powerful than PDA?

A **Turing Machine (TM)** is a mathematical model of general computation.

It consists of:

- finite states
- an infinite/unbounded tape
- a read/write head
- transition rules

Unlike a PDA, a TM can:

- read
- write

-
-

move left

-
-

move right

-
-

repeatedly modify information on the tape

-

Therefore it is much more powerful than a PDA.

For example:

[
{a^n b^n c^n\mid n\geq1}
]

cannot be recognized by a PDA but can be recognized by a TM.

---

13. Define the components of a Turing Machine.

A TM is formally defined as:

[
M=(Q,\Sigma,\Gamma,\delta,q_0,B,F)
]

where:

-

(Q) = finite set of states

-
-

(\Sigma) = input alphabet

-
-

(\Gamma) = tape alphabet

-
-

(\delta) = transition function

-
-

(q_0) = initial state

-
-

(B) = blank symbol

-
-

(F) = set of final states

-

The transition function is:

[
\delta:Q\times\Gamma
\rightarrow
Q\times\Gamma\times{L,R}
]

A transition tells the machine:

1.

what state to enter,

2.
3.

what symbol to write,

4.
5.

which direction to move.

6.

---

14. Explain tape, head, and state transitions in a TM.

Tape

The tape provides the machine's memory.

It contains:

```
... B B 0 1 1 0 B B ...
```

The tape is theoretically unbounded.

Head

The head:

- reads a symbol,
- writes a symbol,
- moves left or right.

State transition

For example:

$$
\delta(q_0,0)=(q_1,X,R)
$$

means:

If the machine is in (q_0) and reads 0, write X, move right, and enter (q_1).

---

15. Explain programming techniques for Turing Machines.

Some common TM programming techniques are:

1. Marking symbols

Replace processed symbols with special symbols.

Example:

[
0\rightarrow X
]

2. Checking off

Mark symbols once they have been matched.

Example:

```
0011
X011
X0Y1
XXY1
XXYY
```

3. Shifting symbols

Move symbols left or right to rearrange the tape.

4. Subroutines

Break a complex algorithm into smaller reusable operations such as:

-

Find left end

-
-

Find next `0`

-
-

Mark symbol

-
-

Return to beginning

-

5. Multiple tracks

Conceptually divide a tape cell into multiple fields to store additional information.

6. Storing information in states

Use finite control to remember small amounts of information.

These techniques make TM designs easier to understand and implement.

---

16. What is storage in finite control in Turing Machines?

Finite control refers to the machine's current state.

A TM can store a **finite amount of information** in its states.

For example:

$$
q_{\text{found0}}
$$

can indicate that the machine has just found a `0`.

Similarly:

$$
q_{\text{carry}}
$$

could represent that the machine is carrying a value during an arithmetic operation.

Limitation

Finite control cannot store an arbitrarily large amount of information.

For large/unbounded information, the TM must use its tape.

So:

$$
\boxed{\text{Finite control = small fixed memory}}
$$

while:

$$
\boxed{\text{Tape = unbounded memory}}
$$

---

17. Compare FA, PDA and TM.

Feature	FA	PDA	TM
Memory	Finite control	Stack	Unbounded tape
Read input	Yes	Yes	Yes
Write memory	No	Stack operations	Yes
Left movement	No	Input normally one-way	Yes
Main language class	Regular	Context-Free	Recursively enumerable
(a^n b^n)	✗	✓	✓
(a^n b^n c^n)	✗	✗	✓
General computation	✗	Limited	✓
Power	Lowest	Medium	Highest

Hierarchy:

$$
\boxed{FA<PDA<TM}
$$

18. What is DFA?

DFA stands for **Deterministic Finite Automaton**.

A DFA is formally:

[
M=(Q,\Sigma,\delta,q_0,F)
]

where:

- (Q) = finite states
- (\Sigma) = input alphabet
- (\delta) = transition function
- (q_0) = initial state
- (F) = final states

The transition function is:

[
\delta:Q\times\Sigma\rightarrow Q
]

Important property

For every state and input symbol, there is **exactly one** next state.

Example:

$$
\delta(q_0,0)=q_1
$$

There is no ambiguity about the next state.

A DFA accepts only **regular languages**.

---

19. Which language is accepted by a Turing Machine?

A Turing Machine can recognize **recursively enumerable languages**.

A language (L) is recursively enumerable if there is a TM that:

- accepts every string belonging to (L),
- may reject or run forever for strings not belonging to (L).

If the TM **halts on every input**, then it is a **decider**, and the language is called **recursive/decidable**.

Therefore, be careful with the wording:

$$
\boxed{\text{TM recognizers accept recursively enumerable languages}}
$$

and:

$$
\boxed{\text{TM deciders decide recursive languages}}
$$

---

20. How are Turing Machines applied in real-world applications?

A Turing Machine is primarily a **theoretical model**, rather than a physical machine used directly in applications.

However, its concepts form the theoretical foundation of modern computing.

Applications/connections include:

1. Algorithm design

TM theory helps define what problems can be solved algorithmically.

2. Compiler design

Compilers perform complex computations involving parsing, transformation and optimization.

3. Complexity theory

TMs are used to study:

- time complexity
- space complexity
- computational limits

4. Computability

TMs help determine whether a problem is computable at all.

5. Program verification

Theoretical computation models are used to study whether programs terminate or satisfy certain properties.

6. Artificial intelligence

Modern computers perform computations that can be modeled by Turing Machines.

7. Cryptography

Algorithms used for encryption, hashing and security can be analyzed using computational models.

Important example

The **Halting Problem** asks whether a program will eventually stop.

Turing showed that there is no general algorithm that can correctly determine this for every possible program and input.

This demonstrates a fundamental limit of computation.

Quick Revision

PDA

$$
\boxed{\text{PDA = Finite Automaton + Stack}}
$$

Recognizes:

$$
\boxed{\text{Context-Free Languages}}
$$

TM

$$
\boxed{\text{TM = Finite Control + Unbounded Read/Write Tape}}
$$

Recognizes:

$$
\boxed{\text{Recursively Enumerable Languages}}
$$

Main hierarchy

$$
\boxed{FA<PDA<TM}
$$

Most important relationships

$$
\boxed{Regular\ Languages\subset Context\text{-}Free\ Languages\subset Recursively\ Enumerable\ Languages}
$$

and:

$$
\boxed{CFG\equiv NPDA}
$$

for language-recognition power.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand